
MAJORITY DYNAMICS IN ONLINE NETWORKS

Reti Sociali



Author

Ferrara Francesco

UNISA

2025-2026

Indice

1	Introduzione	3
1.1	Descrizione del Problema	3
1.2	Obiettivi del Progetto	4
1.3	Strumenti utilizzati	4
2	Metodologia	5
3	La Rete Scelta	6
3.1	Motivazione e Caratteristiche	6
3.2	Exploratory Data Analysis (EDA)	7
4	Configurazione Adottata	9
4.1	Modello di Diffusione e Definizione del Seed Set	9
4.2	Algoritmi di Selezione	10
4.2.1	Algoritmo 1: Cost-Seeds-Greedy	10
4.2.2	Algoritmo 2: Weighted Target Set Selection (WTSS)	12
4.2.3	Algoritmo 3 (Proposto): Real-Gain Greedy	14
4.3	Funzioni di Costo	16
4.3.1	Costo Basato sul Grado	16
4.3.2	Costo Ponte	16
4.4	Impostazione degli Esperimenti	17
5	Esperimenti ed Analisi dei Risultati	18
5.1	Step 1: Utilizzo dei 3 Algoritmi sulla rete G	18
5.2	Step 2: Robustezza alla Rimozione degli Archi	20
5.2.1	Funzione costo : grado	20
5.2.2	Funzione costo : ponte	22
5.3	Step 3: Robustezza alla Rimozione dei Vertici	24
5.3.1	Funzione costo : grado	24
5.3.2	Funzione costo : ponte	26
6	Conclusioni e Sviluppi Futuri	28

1 Introduzione

1.1 Descrizione del Problema

Le reti sociali online possono essere rappresentate mediante grafi, nei quali i vertici rappresentano gli utenti e gli archi rappresentano le relazioni tra essi. Tali reti sono caratterizzate da una natura dinamica: nel tempo gli utenti possono aggiungere o rimuovere relazioni, entrare in nuovi social network oppure abbandonare quelli esistenti. Di conseguenza, anche la struttura della rete può modificarsi nel corso del tempo.

Il progetto considera il problema della *Majority Influence Diffusion*, ovvero un processo di diffusione dell'influenza all'interno di una rete. Data una rete $G = (V, E)$ e un insieme iniziale di nodi, detto *seed set* $S \subseteq V$, il processo di attivazione si sviluppa attraverso una successione di passi temporali. Al tempo iniziale sono attivi esclusivamente i nodi appartenenti a S . Ad ogni passo successivo, un nodo non ancora attivo viene attivato quando almeno metà dei suoi vicini risulta già attiva. Una volta attivato, un nodo rimane attivo per tutti gli step successivi del processo. La diffusione termina quando non è più possibile attivare nuovi nodi; l'insieme finale dei nodi attivati è indicato con $Inf[S]$.

Il problema viene ulteriormente considerato nel caso in cui ad ogni nodo $u \in V$ sia associato un costo $c(u)$ e sia fissato un budget massimo k . Il costo complessivo del seed set è definito come la somma dei costi dei nodi che lo compongono. L'obiettivo consiste quindi nell'individuare un seed set S il cui costo non superi il budget disponibile e che massimizzi il numero di nodi successivamente influenzati, ovvero $|Inf[S]|$.

La determinazione del seed set ottimale è un problema computazionalmente difficile, essendo NP-hard e difficile da approssimare anche nel caso particolare in cui tutti i nodi abbiano lo stesso costo. Per questo motivo, tale lavoro non mira alla ricerca della soluzione ottima, ma alla valutazione sperimentale di algoritmi euristici capaci di determinare un seed set massimale, ossia un insieme di nodi il più grande possibile nel rispetto del budget assegnato.

Un ulteriore aspetto fondamentale del progetto riguarda la natura dinamica delle online network: dopo aver individuato un seed set sulla rete originale G , vengono simulate modifiche alla rete attraverso la rimozione casuale di un certo numero di archi e, separatamente, di un certo numero di vertici. Si ottiene così una nuova rete G' sulla quale viene nuovamente eseguito il processo di diffusione mantenendo invariato il seed set originariamente individuato. Il confronto tra $|Inf[G, S]|$ e $|Inf[G', S]|$ permette di valutare quanto l'efficacia del seed set sia influenzata dai cambiamenti strutturali della rete.

1.2 Obiettivi del Progetto

L'obiettivo principale del progetto è analizzare il comportamento del processo di **Majority Influence Diffusion** in una rete online e valutare l'efficacia di differenti strategie per la selezione del seed set.

In particolare, il progetto prevede l'utilizzo di almeno tre algoritmi di selezione del seed set: due tra quelli proposti nella traccia e un ulteriore algoritmo progettato autonomamente, scelto sulla base della sua possibile efficacia rispetto al problema considerato. Per ciascun algoritmo vengono valutate le dimensioni del seed set e il numero di nodi influenzati al variare del budget k .

L'analisi viene effettuata considerando differenti funzioni di costo associate ai nodi. In particolare, vengono considerate una funzione che assegna costi casuali scelti all'interno di un intervallo prefissato e una funzione basata sul grado del nodo, definita come $c(u) = d(u)/2$. A queste viene aggiunta una terza funzione di costo scelta autonomamente e motivata nell'ambito del progetto.

Un secondo obiettivo consiste nello studiare la robustezza dei seed set rispetto alle modifiche della struttura della rete. A partire dalla rete originale vengono quindi simulate la rimozione casuale di un numero variabile di archi e la rimozione casuale di un numero variabile di vertici. Per ciascuna configurazione viene confrontato il numero di nodi influenzati nella rete originale con quello ottenuto nella rete modificata.

I risultati ottenuti vengono infine rappresentati mediante grafici, con l'obiettivo di confrontare le prestazioni dei diversi algoritmi, analizzare l'influenza del budget sulla diffusione e valutare la robustezza dei seed set rispetto alla perdita di archi e vertici. L'analisi sperimentale permetterà quindi di individuare le strategie più efficaci nelle diverse condizioni considerate.

1.3 Strumenti utilizzati

Per la realizzazione del progetto vengono utilizzati strumenti per la gestione, l'analisi e la visualizzazione delle reti. In particolare, l'implementazione degli algoritmi e delle simulazioni viene realizzata mediante *Python*, che permette di gestire i grafi, implementare il processo di diffusione e automatizzare l'esecuzione degli esperimenti.

Per la rappresentazione e l'analisi dei grafi viene utilizzata la libreria *NetworkX*, mentre *NumPy* e *Pandas* vengono impiegati per la gestione dei dati e dei risultati sperimentali. La produzione dei grafici e la visualizzazione dei risultati vengono realizzate mediante *Matplotlib*. L'ambiente di lavoro utilizzato è *Google Colab*. Gli esperimenti vengono eseguiti considerando la rete scelta per il progetto e differenti configurazioni dei parametri, tra cui il budget del seed set, la funzione di costo e il numero di archi o vertici rimossi. I risultati vengono successivamente raccolti e confrontati attraverso rappresentazioni grafiche, in modo da rendere possibile una valutazione sistematica delle prestazioni degli algoritmi considerati.

2 Metodologia

Il lavoro in esame è iniziato con la ricerca e la selezione della rete sfruttando la piattaforma **SNAP**, scelta per l'ampia disponibilità di grafi di varia natura. Successivamente si è provveduto all'EDA, la fase in cui la rete viene sottoposta ad una "lettura" per comprenderne le caratteristiche specifiche.

A partire da questa analisi preliminare è stato definito un modello di diffusione dell'influenza a soglia, insieme alla formalizzazione del problema di selezione del seed set sotto vincolo di budget. Sono stati quindi implementati tre algoritmi di selezione, due tratti dalla letteratura di riferimento (Cost-Seeds-Greedy e Weighted Target Set Selection) e un terzo, Real-Gain Greedy, proposto come variante che sostituisce la stima approssimata del guadagno con il valore reale ottenuto tramite simulazione diretta del processo di diffusione. Per rendere quest'ultimo computazionalmente sostenibile è stata inoltre introdotta una strategia di riduzione dello spazio dei candidati basata su community detection.

Il confronto tra i tre algoritmi è stato condotto rispetto a due funzioni di costo differenti, una basata sul grado del nodo e una basata sul suo ruolo strutturale di "ponte" tra community, in modo da valutare la sensibilità di ciascun algoritmo a diverse assunzioni sul costo di attivazione dei nodi.

Il lavoro sperimentale è stato articolato in tre fasi successive:

- nella prima fase (Step 1) i tre algoritmi sono stati eseguiti sulla rete originale G , per due valori di budget, misurando la dimensione del seed set individuato e l'estensione della cascata di diffusione risultante;
- nella seconda fase (Step 2) è stata valutata la robustezza dei seed set individuati rispetto alla rimozione casuale di una porzione degli archi della rete, confrontando l'influenza ottenuta sulla rete perturbata con quella di riferimento sulla rete originale;
- nella terza fase (Step 3) la stessa analisi di robustezza è stata ripetuta rimuovendo, anziché archi, una porzione di vertici della rete (esclusi quelli appartenenti al seed set), per valutare la resilienza dei risultati anche a perturbazioni di natura strutturale più drastica.

3 La Rete Scelta

3.1 Motivazione e Caratteristiche

Il grafo preso in esame, disponibile presso il dataset SNAP - Social circles: Facebook, riguarda una rete sociale costruita a partire da 10 *ego-network* basate sul social network Facebook. Il dataset contiene complessivamente 4.039 nodi e 88.234 archi ed è stato ottenuto a partire dai dati relativi a 10 utenti Facebook, per i quali sono state raccolte le rispettive reti di relazioni.

Una *ego-network* rappresenta la rete sociale osservata dal punto di vista di uno specifico utente, detto *ego*. Essa comprende l'ego stesso, i suoi contatti diretti, detti *alter*, e le relazioni presenti tra tali utenti. Nel caso del dataset considerato, ciascuna delle 10 *ego-network* è quindi centrata su uno specifico utente Facebook e comprende le sue connessioni sociali.

Le dieci *ego-network* vengono successivamente combinate in un'unica rete, mantenendo gli identificativi dei nodi coerenti tra le diverse reti. L'operazione non implica tuttavia che i dieci utenti centrali siano necessariamente collegati direttamente tra loro: la rete risultante è semplicemente ottenuta dall'unione delle relazioni presenti nelle singole *ego-network*. Le diverse reti possono inoltre presentare nodi in comune, consentendo alle rispettive strutture di sovrapporsi.

Il dataset risultante è rappresentato come un grafo non orientato, nel quale un arco indica una relazione di amicizia tra due utenti. La rete combinata risulta connessa: tutti i 4.039 nodi appartengono infatti alla componente connessa principale. Tra le principali caratteristiche riportate da SNAP si trovano un coefficiente medio di clustering pari a 0.6055, 1.612.010 triangoli e un diametro pari a 8.

3.2 Exploratory Data Analysis (EDA)

Prima di procedere con il progetto vero e proprio sulla rete, mi soffermerò sull'analisi esplorativa iniziale, per comprendere che tipo di rete si ha di fronte e come la sua struttura può impattare sulle fasi successive.

Tipo del Dato	Valore del Dato
Grado medio	43.69
Grado mediano	25
Grado massimo	1045
Coefficiente di clustering medio	0.61
Densità	0.011
Numero componenti connesse	1
Numero di nodi della componente principale	4039
Distanza media stimata	3.72
Diametro stimato	8

Table 1: Dati generali sulla rete

La tabella risultante rivela la concordanza tra i dati riscontrati dall'EDA e quelli forniti su SNAP.

In base a questi valori è possibile osservare il fenomeno dello Small-World, il quale è caratterizzato dalla compresenza di due proprietà: una distanza media contenuta tra i nodi e un coefficiente di clustering significativamente più alto di quello atteso in un grafo casuale di pari densità. La rete analizzata soddisfa entrambe le condizioni: la distanza media stimata (3.72) e il diametro stimato (8) mostrano che è possibile raggiungere qualsiasi nodo con pochi salti nonostante le dimensioni della rete (4039 nodi); allo stesso tempo, il coefficiente di clustering osservato (0.61) è drasticamente superiore a quello atteso in un grafo Erdős-Rényi con la stessa densità (≈ 0.011),

$$p = \frac{m}{\binom{n}{2}} = \frac{2m}{n(n-1)} = \frac{2 \cdot 88234}{4039(4039-1)} \approx 0.01082 \quad (1)$$

Figure 1: Coefficiente di Clustering in un grafo Erdős-Rényi di pari dimensioni.

a conferma della presenza di una forte struttura locale tipica delle reti sociali reali. C'è un'altra considerazione importante da fare, che riguarda il fatto che il grafo selezionato è rappresentato da un'unica componente connessa, di conseguenza non ci sono "buchi" o separazioni. Questo è dovuto al fatto che le 10 ego-network condividono una parte dei propri nodi (alters): gli amici di un utente ego compaiono spesso anche nelle cerchie di altri utenti ego del campione, e questa sovrapposizione naturale collega le sotto-reti in un'unica componente, senza la necessità di introdurre archi artificiali per forzare la connessione.

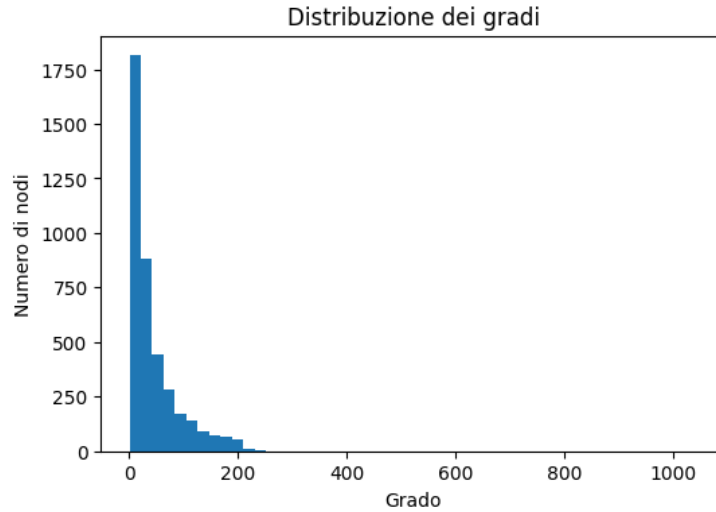


Figure 2: Distribuzione dei gradi

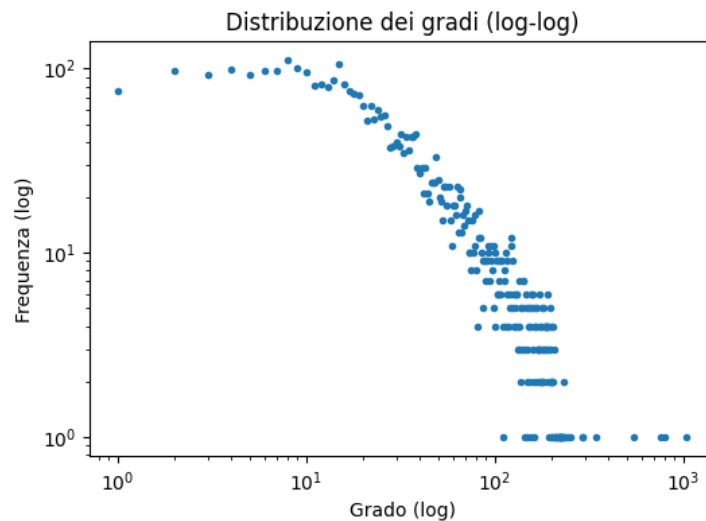


Figure 3: Distribuzione dei gradi (log-log)

Andando a plottare i gradi si osserva un andamento simile ad una power law, che è una funzione tipica delle reti sociali, ossia nonostante la grandezza della rete, pochissimi nodi hanno un grado elevato, mentre molti hanno un grado molto basso. Essendo la rete l'unione di più ego-network, è possibile osservare nel secondo grafico la presenza di scalini: questo è tipico proprio quando **si fondono insieme più ego-network**: ognuna delle 10 sotto-reti ha una sua scala tipica di grado (dovuta alla dimensione della cerchia di amici di quell'ego), e sommandole si ottiene una distribuzione che è la **sovrapposizione di più funzioni**, non un singolo processo power-law puro.

4 Configurazione Adottata

4.1 Modello di Diffusione e Definizione del Seed Set

Formalizziamo in questa sezione il modello di diffusione introdotto nel Capitolo 1, insieme alla notazione utilizzata nel seguito della relazione.

Data una rete $G = (V, E)$ e un seed set $S \subseteq V$, il processo di **Majority Influence Diffusion** è definito come una successione di sottoinsiemi $Inf[S, 0], Inf[S, 1], \dots, Inf[S, r], \dots \subseteq V$, dove:

$$Inf[S, 0] = S$$

$$Inf[S, r] = Inf[S, r-1] \cup \{v \in V - Inf[S, r-1] : |N(v) \cap Inf[S, r-1]| \geq d(v)/2\}$$

Il più piccolo istante t tale che $Inf[S, t] = Inf[S, t+1]$ rappresenta il momento in cui la cascata si arresta, e si pone $Inf[S] = Inf[S, t]$. Si osservi che, poiché ad ogni passo i nuovi nodi attivati vengono scelti in $V - Inf[S, r-1]$, un nodo che si attiva rimane attivo per tutti gli step successivi del processo.

Nel caso in cui ad ogni nodo $u \in V$ sia associato un costo $c(u)$, il costo di un seed set S è definito come $c(S) = \sum_{u \in S} c(u)$. Fissato un budget k , si definisce *seed set massimale* il più grande insieme S , secondo il criterio adottato dall'algoritmo di selezione utilizzato, tale che $c(S) \leq k$. È importante distinguere il seed set massimale dal seed set *ottimo*, ovvero quello che massimizzerebbe $|Inf[S]|$ nel rispetto del vincolo di budget: quest'ultimo non viene ricercato, data la natura NP-hard del problema discussa nel Capitolo 1. Il seed set massimale rappresenta invece l'output prodotto da un algoritmo euristico, e il suo criterio di costruzione varia da algoritmo ad algoritmo. Proprio per questo motivo, il confronto sperimentale tra più algoritmi di selezione, presentato nei capitoli successivi, permette di valutare quale strategia di riempimento del budget risulti più efficace in termini di nodi effettivamente influenzati.

4.2 Algoritmi di Selezione

4.2.1 Algoritmo 1: Cost-Seeds-Greedy

```
def cost_seeds_greedy(G, k, cost, thresholds=None):  
  
    if thresholds is None:  
        thresholds = compute_thresholds(G)  
  
    cnt = dict.fromkeys(G.nodes(), 0)  # |N(v) ∩ S| corrente  
    S = set()  
    cost_S = 0  
    candidates = set(G.nodes())  
  
    while True:  
        best_v, best_ratio = None, -1  
        for v in candidates:  
            gain = sum(1 for u in G.neighbors(v) if cnt[u] < thresholds[u])  
            ratio = gain / cost[v]  
            if ratio > best_ratio:  
                best_ratio, best_v = ratio, v  
  
        if best_v is None:  
            break  
  
        new_cost = cost_S + cost[best_v]  
        if new_cost > k:  
            break  
  
        S.add(best_v)  
        candidates.discard(best_v)  
        cost_S = new_cost  
        for u in G.neighbors(best_v):  
            cnt[u] += 1  
  
    return S
```

Figure 4: Algoritmo 1: Cost-Seeds-Greedy

Il primo algoritmo considerato, denominato *Cost-Seeds-Greedy*, costruisce il seed set in maniera incrementale, selezionando ad ogni iterazione il nodo che offre il miglior rapporto tra guadagno marginale e costo.

Dato un intero k , una funzione costi $c : V \rightarrow \mathbb{N}$ e una funzione f_i (con $i = 1, 2, 3$), l'algoritmo mantiene due insiemi S_p e S_d , inizialmente vuoti. Ad ogni iterazione viene selezionato il nodo

$$u = \arg \max_{v \in V - S_d} \frac{\Delta_v f_i(S_d)}{c(u)}$$

dove $\Delta_v f_i(S_d)$ rappresenta il guadagno marginale ottenuto aggiungendo il nodo v all'insieme S_d secondo la funzione f_i . Il nodo selezionato viene quindi aggiunto a S_d , mentre S_p conserva lo stato precedente all'inserimento. L'iterazione prosegue finché il costo di S_d non supera il budget k ; a quel punto l'algoritmo restituisce S_p , ovvero l'ultimo insieme che rispetta il vincolo di budget.

Le funzioni f_i considerate approssimano il numero di nodi che ciascun vertice contribuisce ad attivare, sulla base del numero di vicini appartenenti al seed set corrente:

$$\begin{aligned}
f_1(S) &= \sum_{v \in V} \min \left\{ |N(v) \cap S|, \left\lceil \frac{d(v)}{2} \right\rceil \right\} \\
f_2(S) &= \sum_{v \in V} \sum_{i=1}^{|N(v) \cap S|} \max \left\{ \left\lceil \frac{d(v)}{2} \right\rceil - i + 1, 0 \right\} \\
f_3(S) &= \sum_{v \in V} \sum_{i=1}^{|N(v) \cap S|} \max \left\{ \frac{\lceil d(v)/2 \rceil - i + 1}{d(v) - i + 1}, 0 \right\}
\end{aligned}$$

Nel presente lavoro è stata adottata la funzione f_1 . La scelta è motivata da due considerazioni. Da un punto di vista concettuale, f_1 fornisce la stima più diretta del contributo di un nodo all'attivazione: per ciascun vertice v conta il numero di vicini attivi fino al raggiungimento della soglia $\lceil d(v)/2 \rceil$, senza attribuire ulteriore valore ai vicini eccedenti la soglia stessa. Le funzioni f_2 e f_3 estendono invece il conteggio anche oltre la soglia, assegnando un peso decrescente ai contributi successivi; tale raffinamento risulta però meno immediato da interpretare nel contesto del progetto, poiché introduce un ordinamento implicito tra i vicini di S che non trova un corrispettivo diretto nel processo di diffusione reale, in cui l'attivazione di un nodo dipende unicamente dal superamento della soglia e non dall'ordine con cui i vicini si sono attivati.

Da un punto di vista implementativo, inoltre, f_1 richiede unicamente il conteggio dei vicini attivi di ciascun nodo, mentre f_2 e f_3 richiedono di tracciare l'ordine di inserimento dei vicini in S per calcolare correttamente la doppia sommatoria, con un conseguente aumento della complessità computazionale e della probabilità di errori in fase di implementazione.

4.2.2 Algoritmo 2: Weighted Target Set Selection (WTSS)

```
def wtss(G, k, cost, thresholds=None):

    if thresholds is None:
        thresholds = compute_thresholds(G)

    delta = dict(G.degree())          #  $\delta(v)$ : grado residuo nel grafo U
    kk = dict(thresholds)             #  $k(v)$ : soglia residua
    N = {v: set(G.neighbors(v)) for v in G.nodes()} # vicinato residuo
    U = set(G.nodes())
    S = set()
    cost_S = 0

    while U:
        # Caso 1: esiste v in U con  $k(v) = 0$ 
        v1 = next((v for v in U if kk[v] == 0), None)

        if v1 is not None:
            v = v1
            for u in N[v]:
                kk[u] = max(kk[u] - 1, 0)
        else:
            # Caso 2: esiste v in U con  $\delta(v) < k(v)$ 
            v2 = next((v for v in U if delta[v] < kk[v]), None)

            if v2 is not None:
                v = v2
                new_cost = cost_S + cost[v]
                if new_cost > k:
                    break # budget superato: fermo qui, S rispetta ancora k
                S.add(v)
                cost_S = new_cost
                for u in N[v]:
                    kk[u] -= 1
            else:
                # Caso 3: nessuno dei precedenti, si sceglie per rapporto
                v = max(U, key=lambda u: (cost[u] * kk[u]) / (delta[u] * (delta[u] + 1)))

        # Rimozione di v dal grafo residuo (comune a tutti e tre i casi)
        for u in N[v]:
            delta[u] -= 1
        N[v].discard(v)
        U.discard(v)

    return S
```

Figure 5: Algoritmo 2: WTSS

Il secondo algoritmo considerato è *WTSS* (Weighted Target Set Selection), il quale determina il seed set massimale eliminando iterativamente i nodi dal grafo secondo un criterio basato sulla soglia di attivazione residua di ciascun vertice.

Fissato il budget k , si assegna ad ogni nodo u una soglia $t(u) = d(u)/2$ e un costo $c(u)$. L'algoritmo mantiene, per ogni nodo v ancora presente nel grafo residuo U , un grado corrente $\delta(v)$ e una soglia residua $k(v)$, inizializzati rispettivamente a $d_G(v)$ e $t(v)$.

Ad ogni iterazione viene selezionato ed eliminato un nodo secondo uno dei seguenti tre casi:

- **Caso 1:** se esiste $v \in U$ con $k(v) = 0$, il nodo risulta già attivabile tramite i vicini esterni a U ; viene quindi rimosso dal grafo e la soglia residua dei suoi vicini in U viene decrementata di uno (senza scendere sotto zero).
- **Caso 2:** se esiste $v \in U$ con $\delta(v) < k(v)$, il numero di vicini rimasti in U non è sufficiente a soddisfare la soglia residua di v ; il nodo viene quindi inserito nel seed set S , e la soglia residua dei suoi vicini viene decrementata di uno.
- **Caso 3:** in tutti gli altri casi, viene selezionato il nodo $v = \arg \max_{u \in U} \frac{c(u)k(u)}{\delta(u)(\delta(u)+1)}$, che verrà attivato dai propri vicini in U .

In tutti i casi, il nodo selezionato viene rimosso dal grafo residuo aggiornando il grado $\delta(u)$ e il vicinato $N(u)$ dei nodi rimanenti. L'algoritmo termina quando $U = \emptyset$, restituendo il seed set S costruito.

A differenza di Cost-Seeds-Greedy, WTSS non richiede di fissare a priori un limite di iterazioni legato al budget k : il criterio di arresto è intrinseco al processo di eliminazione dei nodi dal grafo, e il vincolo di costo interviene nella scelta del nodo tramite il rapporto $c(u)k(u)/(\delta(u)(\delta(u) + 1))$ nel Caso 3.

4.2.3 Algoritmo 3 (Proposto): Real-Gain Greedy

```
def real_gain_greedy(G, k, cost, pool, thresholds=None):

    if thresholds is None:
        thresholds = compute_thresholds(G)

    S = set()
    cost_S = 0
    candidates = set(pool)
    Inf_S = simula_diffusione(G, S, thresholds)

    while True:
        best_v, best_ratio, best_Inf = None, -1, None

        for v in candidates:
            residual = k - cost_S
            if cost[v] > residual:
                continue # scarta i candidati non fattibili, non interrompe il ciclo

            Inf_new = simula_diffusione(G, S | {v}, thresholds)
            gain = len(Inf_new) - len(Inf_S)
            ratio = gain / cost[v]
            if ratio > best_ratio:
                best_ratio, best_v, best_Inf = ratio, v, Inf_new

        if best_v is None:
            break # nessun candidato fattibile rimasto nel pool: fine

        S.add(best_v)
        candidates.discard(best_v)
        cost_S += cost[best_v]
        Inf_S = best_Inf

    return S
```

Figure 6: Algoritmo 3: Real-Gain Greedy

Il terzo algoritmo, denominato *Real-Gain Greedy*, adotta uno schema incrementale analogo a Cost-Seeds-Greedy, ma sostituisce la stima approssimata fornita dalla funzione f_i con il guadagno reale ottenuto simulando effettivamente il processo di diffusione. Ad ogni iterazione viene selezionato il nodo

$$u = \arg \max_{v \in P_d} \frac{|Inf[S_d \cup \{v\}]| - |Inf[S_d]|}{c(v)}$$

dove $|Inf[S_d \cup \{v\}]|$ è calcolato facendo girare per intero il processo di attivazione descritto nella sottosezione 4.1, e $P_d \subseteq V$ è l'insieme residuo dei candidati ammissibili al passo d (si veda oltre).

A differenza di Cost-Seeds-Greedy, che stima il contributo di un nodo tramite una funzione di supporto calcolabile in tempo costante rispetto al vicinato, Real-Gain Greedy calcola il guadagno marginale esatto rispetto all'obiettivo reale $|Inf[S]|$, al costo di una simulazione completa della diffusione per ciascun candidato non ancora incluso in S_d . Questo algoritmo viene incluso nel confronto sperimentale per valutare quanto la stima approssimata f_1 si discosti dal guadagno reale ottenibile con un approccio esaustivo ma computazionalmente più oneroso.

```

def build_diverse_pool(G, communities, scala='sqrt', min_per_community=1):

    n = G.number_of_nodes()
    if scala == 'sqrt':
        target_size = max(1, round(math.sqrt(n)))
    elif scala == 'log':
        target_size = max(1, round(math.log(n) * 10)) # fattore di scala da tarare
    else:
        raise ValueError("scala deve essere 'sqrt' o 'log'")

    pool = set()
    for community in communities:
        quota = max(min_per_community, round(target_size * len(community) / n))
        top_in_community = sorted(community, key=lambda v: G.degree(v), reverse=True)[:quota]
        pool.update(top_in_community)

    return pool

```

Figure 7: Pool utilizzato precedentemente all'algoritmo 3

Pool di candidati diversificato per community. Data la complessità computazionale della simulazione, Real-Gain Greedy non opera sull'intero insieme V ma su un pool ridotto di candidati $P \subseteq V$, costruito a priori.

Il pool viene costruito a partire da una partizione in community della rete, ottenuta tramite l'algoritmo di Louvain, e includendo per ciascuna community una quota di nodi ad alto grado proporzionale alla sua dimensione relativa $|C_i|/|V|$. L'algoritmo di Louvain identifica nella rete G un numero di community pari a 16, di cui la più grande ha una dimensione pari a 548, mentre le due più piccole di 19.

La dimensione complessiva del pool è mantenuta sub-lineare rispetto a $|V|$ (per default $|P| \approx \sqrt{|V|}$, anziché una percentuale fissa tarata sulla singola rete), in modo da preservare la scalabilità del metodo. Questa costruzione garantisce che il pool rappresenti l'intera struttura comunitaria della rete, e non solo le sue regioni a densità di grado più elevata.

Condizione di arresto. A differenza di Cost-Seeds-Greedy che si ferma al primo candidato non conveniente in ordine di best-ratio, Real-Gain Greedy prosegue la ricerca tra i candidati rimasti nel pool residuo P_d , scartando solamente quelli non ammissibili rispetto al budget residuo ($c(v) > k - c(S_d)$), e si arresta soltanto quando nessun candidato del pool risulta più ammissibile. Questa scelta è motivata dal fatto che, operando su un pool ridotto anziché sull'intero V , fermarsi al primo candidato non conveniente in ordine di best-ratio produrrebbe seed set fortemente sub-ottimali.

4.3 Funzioni di Costo

4.3.1 Costo Basato sul Grado

La seconda funzione di costo lega il costo di attivazione di un nodo alla sua soglia di attivazione nel processo di majority dynamics:

$$c(u) = \left\lceil \frac{d(u)}{2} \right\rceil$$

La scelta riflette l'idea che convincere direttamente un nodo con molte connessioni, e quindi con una soglia di attivazione elevata, richieda uno sforzo maggiore rispetto a un nodo periferico con pochi vicini. Questa funzione introduce quindi una correlazione diretta tra il costo di un nodo e la sua difficoltà di attivazione indiretta.

4.3.2 Costo Ponte

La funzione di costo ponte assegna a ciascun nodo $u \in V$ un valore proporzionale al suo ruolo strutturale di collegamento tra regioni diverse della rete, anziché a una caratteristica puramente locale come il grado. L'intuizione è che i nodi con basso coefficiente di clustering locale, ossia pochi triangoli tra i propri vicini, tendono a comportarsi da *weak ties*, collegando community altrimenti poco connesse tra loro. Tali nodi sono considerati più costosi da attivare, in quanto strategicamente più difficili da raggiungere, ma potenzialmente più efficaci nel favorire la diffusione tra cluster diversi una volta attivati. Formalmente, il costo del nodo u è definito come

$$c(u) = \max \left(1, \left\lceil \frac{d(u)}{2} \right\rceil \cdot (1 - cc(u)) \right)$$

dove $d(u)$ è il grado del nodo e $cc(u) \in [0, 1]$ il suo coefficiente di clustering locale. Il termine $\lceil d(u)/2 \rceil$ fissa il costo massimo raggiungibile, rendendolo comparabile in scala con quello adottato dalla funzione costo-grado, mentre il fattore $(1 - cc(u))$ modula tale valore in funzione dell'isolamento strutturale del nodo: un nodo con $cc(u) = 0$ (nodo ponte "puro", i cui vicini non sono mai collegati tra loro) raggiunge il costo massimo, mentre un nodo fortemente inserito in un triangolo denso di vicini ($cc(u) \rightarrow 1$) tende al costo minimo pari a 1.

A differenza della funzione costo-grado, che penalizza semplicemente i nodi più connessi, la funzione costo ponte introduce quindi una dipendenza dalla topologia locale della rete, distinguendo tra nodi ad alto grado ma strutturalmente "chiusi" all'interno di una community e nodi che, a parità di grado, fungono da connettori tra community diverse.

4.4 Impostazione degli Esperimenti

Per ciascuna delle due funzioni di costo considerate sono stati eseguiti i tre algoritmi di selezione del seed set, facendo variare il budget k su due valori, $k = 500$ e $k = 1000$, scelti per rappresentare rispettivamente uno scenario di selezione più ristretta e uno scenario di copertura più ampia della rete.

Per ogni algoritmo e ogni valore di k è stato calcolato il seed set S e il corrispondente $|Inf[G, S]|$. Poiché il processo di diffusione è deterministico a parità di G , S e soglie di attivazione, questo calcolo è stato eseguito una sola volta per ciascuna combinazione di algoritmo, funzione di costo e budget.

Per la valutazione della robustezza, a partire dalla rete originale G sono stati generati grafi modificati G' rimuovendo casualmente, rispettivamente, una percentuale $p \in \{5\%, 20\%\}$ degli archi e, separatamente, la stessa percentuale dei vertici di G . Nel caso della rimozione di archi, il pool di elementi rimovibili coincide con l'intero insieme E , inclusi gli archi incidenti ai nodi del seed set S ; nel caso della rimozione di vertici, invece, i nodi di S sono esclusi dal pool e non possono mai essere rimossi, poiché la loro eliminazione renderebbe il confronto con $|Inf[G, S]|$ privo di significato. Su ciascun grafo modificato G' è stato mantenuto invariato il seed set S individuato sulla rete originale e ricalcolato $|Inf[G', S]|$, confrontandolo poi con $|Inf[G, S]|$.

5 Esperimenti ed Analisi dei Risultati

5.1 Step 1: Utilizzo dei 3 Algoritmi sulla rete G

Di seguito vengono mostrati i grafici relativi allo Step 1. La visualizzazione consente la divisione per ogni tipo di test effettuato:

1. Funzione di costo(grado o ponte)
2. K utilizzato (500/1000)
3. Algoritmo utilizzato (CSG, WTSS, RGG)
4. Seed set S iniziale (in blu) vs insieme di nodi raggiunti $Inf[G, S]$ (in rosso)

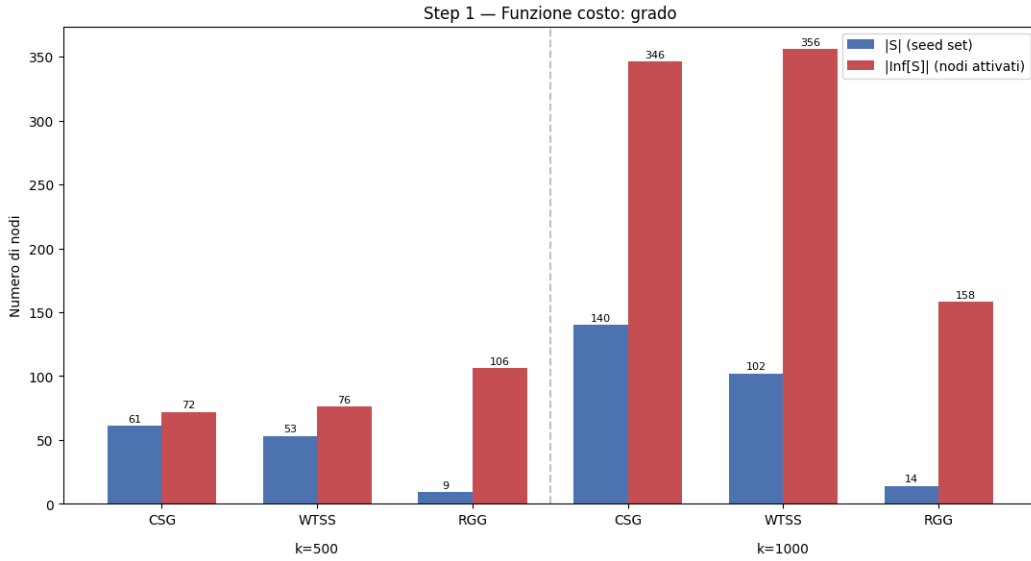


Figure 8: Performance degli algoritmi basata sulla funzione costo grado

Funzione costo-grado. A parità di budget, i tre algoritmi mostrano comportamenti nettamente distinti. RGG seleziona insiemi di seed molto più piccoli rispetto a CSG e WTSS (9 nodi contro 61 e 53 a $k = 500$; 14 contro 140 e 102 a $k = 1000$), coerentemente con il fatto di operare su un pool ristretto di candidati ad alto grado anziché sull'intero V . A $k = 500$ questa selettività si traduce in un'efficienza per nodo superiore: RGG ottiene $|Inf[S]| = 106$ con soli 9 seed, contro i 72 e 76 nodi attivati rispettivamente da CSG e WTSS con seed set 6-7 volte più grandi.

A $k = 1000$, tuttavia, il quadro si inverte in termini assoluti: mentre CSG e WTSS, potendo attingere a tutto V , sfruttano il budget aggiuntivo per quasi raddoppiare l'insieme di semi e raggiungere rispettivamente 346 e 356 nodi attivati, RGG cresce solo marginalmente (14 seed, $|Inf[S]| = 158$), segnalando che il pool costruito su base community diventa un collo di bottiglia a budget elevati: i candidati di qualità nel pool si esauriscono prima che il budget venga sfruttato appieno.

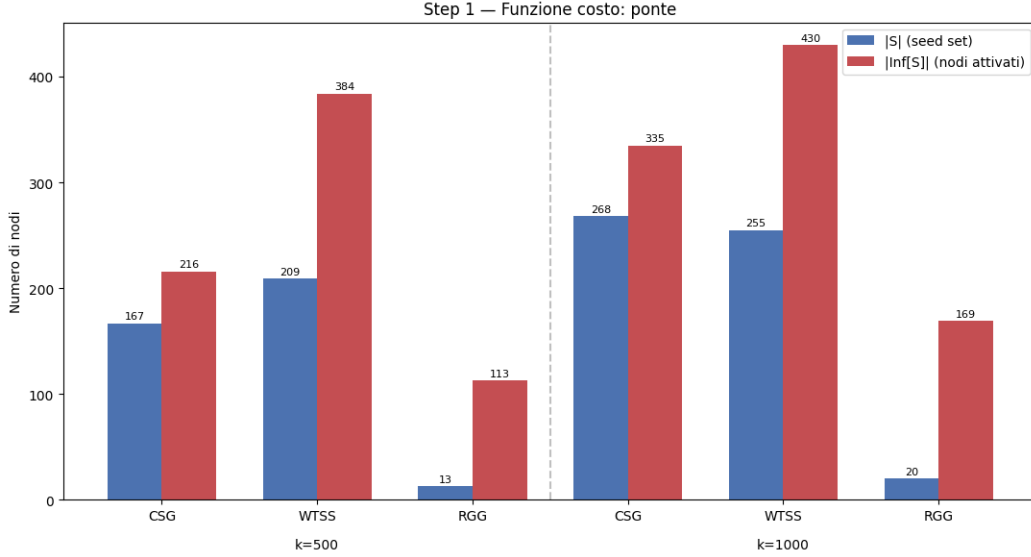


Figure 9: Performance degli algoritmi basata sulla funzione costo ponte

Funzione costo-ponte. Con la funzione costo-ponte, che penalizza maggiormente i nodi strutturalmente "ponte" (basso clustering), i seed set di CSG e WTSS crescono sensibilmente rispetto al caso costo-grado, a causa della diversa distribuzione dei costi nella rete: CSG passa da 61 a 167 seed a $k = 500$ (da 140 a 268 a $k = 1000$), WTSS da 53 a 209 (da 102 a 255). WTSS si conferma l'algoritmo più efficace in termini di copertura assoluta, raggiungendo $|Inf[S]| = 384$ a $k = 500$ e 430 a $k = 1000$, superando sia CSG (216 e 335) sia, in misura più marcata, RGG (113 e 169). RGG mantiene anche in questo scenario la caratteristica di seed set molto più contenuti (13 e 20 nodi), ma senza il vantaggio di efficienza osservato a $k = 500$ con la costo-grado: il rapporto $|Inf[S]|/|S|$ resta comunque il più alto tra i tre algoritmi, a conferma che la selettività di RGG produce seed individualmente più efficaci, anche se la dimensione ridotta del pool ne limita la capacità di scalare l'influenza complessiva all'aumentare del budget.

5.2 Step 2: Robustezza alla Rimozione degli Archi

5.2.1 Funzione costo : grado

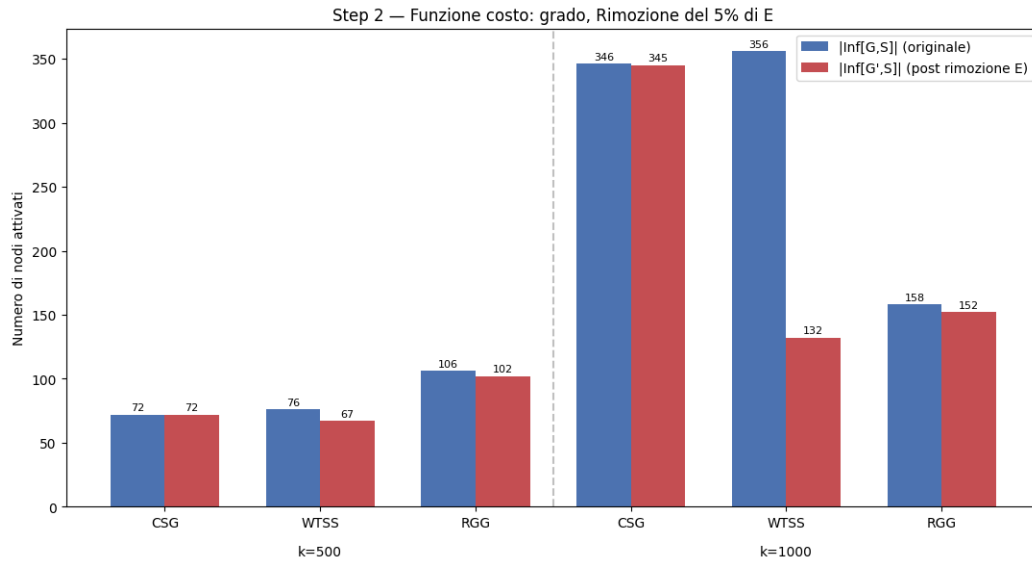


Figure 10: Performance degli algoritmi basata sulla funzione costo grado e sulla rimozione del 5 % di archi

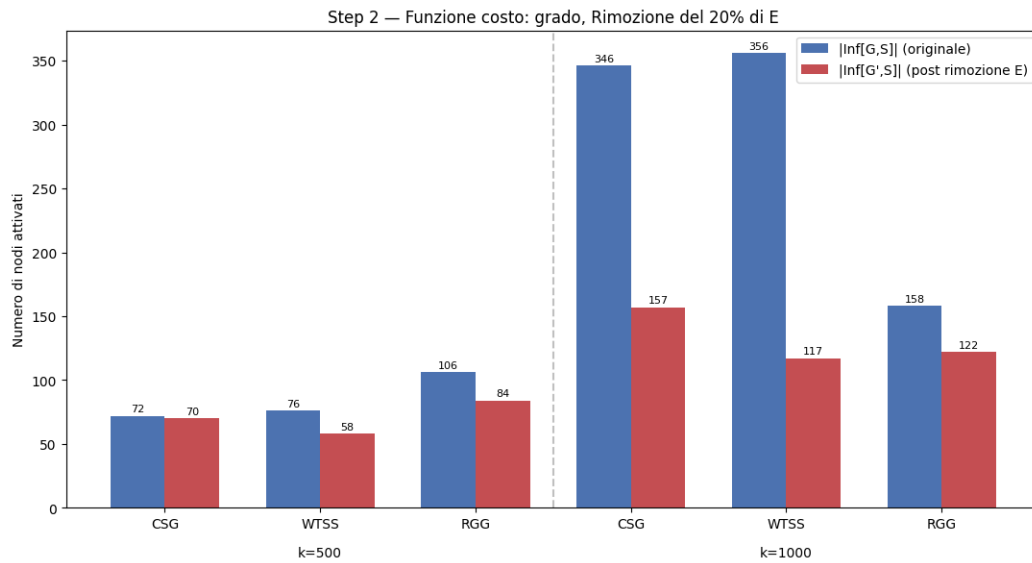


Figure 11: Performance degli algoritmi basata sulla funzione costo grado e sulla rimozione del 20 % di archi

La rimozione del 5% degli archi ha un impatto quasi trascurabile su CSG (variazione nulla o minima, -0.3% a $k = 1000$), mentre WTSS mostra già a questo livello una perdita rilevante (-11.8% a $k = 500$, -62.9% a $k = 1000$). Con il 20% di archi rimossi il divario si accentua drasticamente: a $k = 1000$, CSG riduce da 346 a 157 nodi attivati (-54.6%) e WTSS da 356 a 117 (-67.1%), mentre RGG, pur partendo da un $|Inf[G, S]|$ assoluto molto più basso (158), perde solo il 22.8%. Questo comportamento è coerente con quanto osservato sul grado residuo dei nodi in S dopo la modifica della rete: i seed di RGG, selezionati da un pool orientato a nodi ad alto grado diversificati per community, mantengono un grado medio residuo molto più alto (fino a 134.6 a $k = 1000$) rispetto a quelli di CSG e WTSS (rispettivamente 11.4 e 18.1), risultando quindi strutturalmente meno vulnerabili alla rimozione casuale di archi.

5.2.2 Funzione costo : ponte

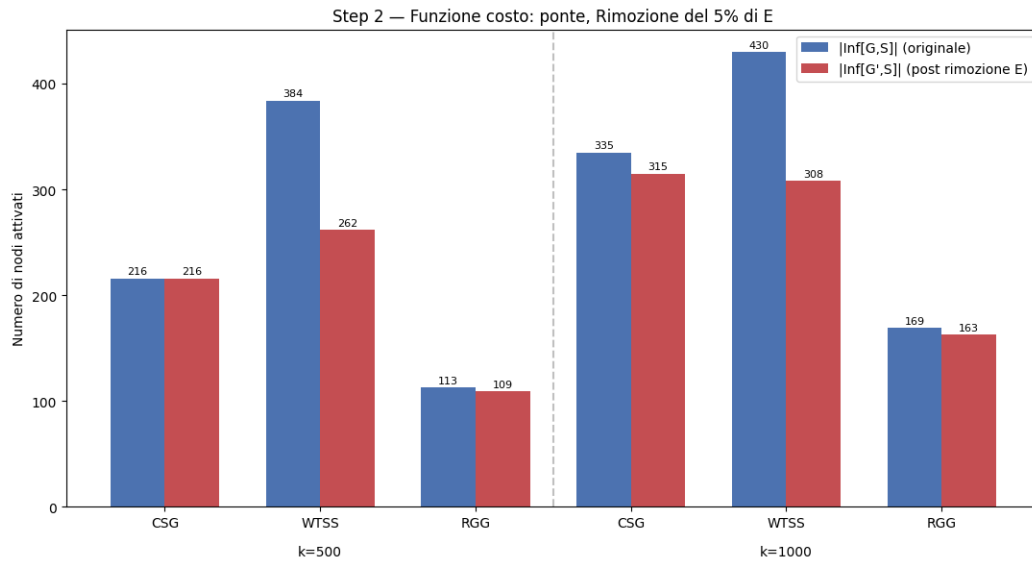


Figure 12: Performance degli algoritmi basata sulla funzione costo ponte e sulla rimozione del 5 % di archi

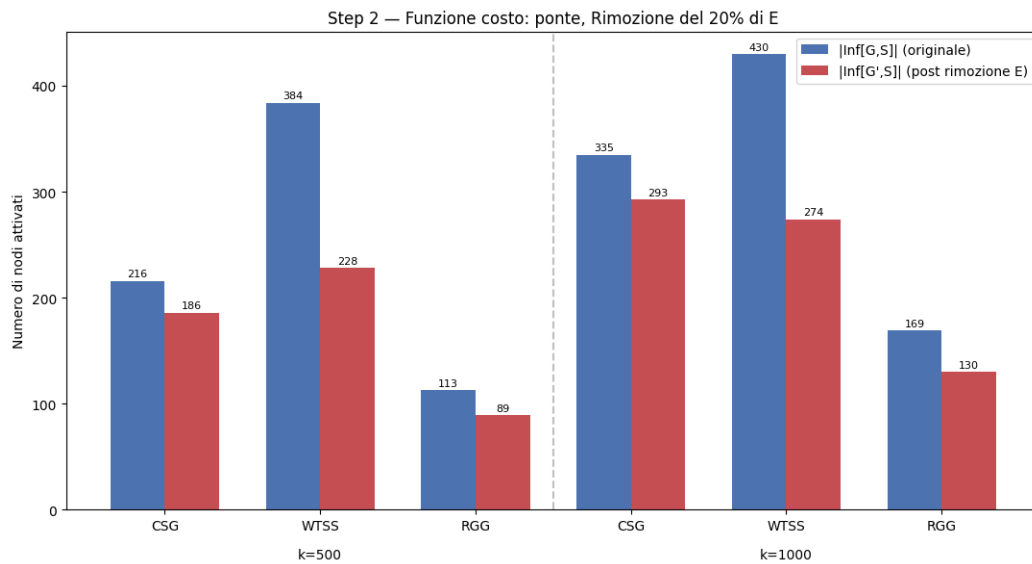


Figure 13: Performance degli algoritmi basata sulla funzione costo ponte e sulla rimozione del 20 % di archi

Il quadro qualitativo si conferma anche con la funzione costo-ponte, sebbene con un'entità delle perdite complessivamente inferiore rispetto al caso costo-grado. WTSS resta l'algoritmo più fragile: a $k = 500$ perde già il 31.8% di $|Inf[G, S]|$ con la sola rimozione del 5% degli archi, arrivando al 40.6% al 20%; un pattern simile si osserva a $k = 1000$ (−28.4% e −36.3%). CSG si comporta in modo più stabile, specialmente a $k = 500$ (perdita nulla al 5%, −13.9% al 20%). RGG, pur mostrando in valore assoluto gli $|Inf[G, S]|$ più contenuti (113 e 169), è ancora una volta l'algoritmo più resiliente in termini percentuali, con perdite comprese tra il 3.5% e il 23.1% anche al livello di perturbazione più severo. Il risultato conferma quanto osservato con la funzione costo-grado: la selettività di RGG nello scegliere seed strutturalmente centrali all'interno di pool diversificati per community si traduce in una maggiore robustezza della cascata di diffusione rispetto a una perdita casuale di connettività, indipendentemente dalla funzione di costo adottata.

5.3 Step 3: Robustezza alla Rimozione dei Vertici

5.3.1 Funzione costo : grado

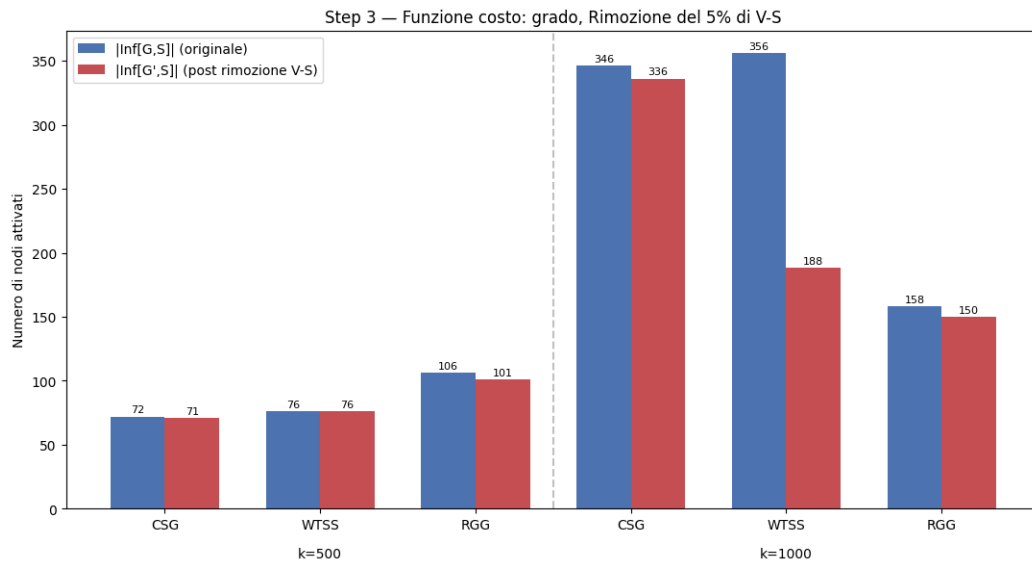


Figure 14: Performance degli algoritmi basata sulla funzione costo grado e sulla rimozione del 5 % di V-S

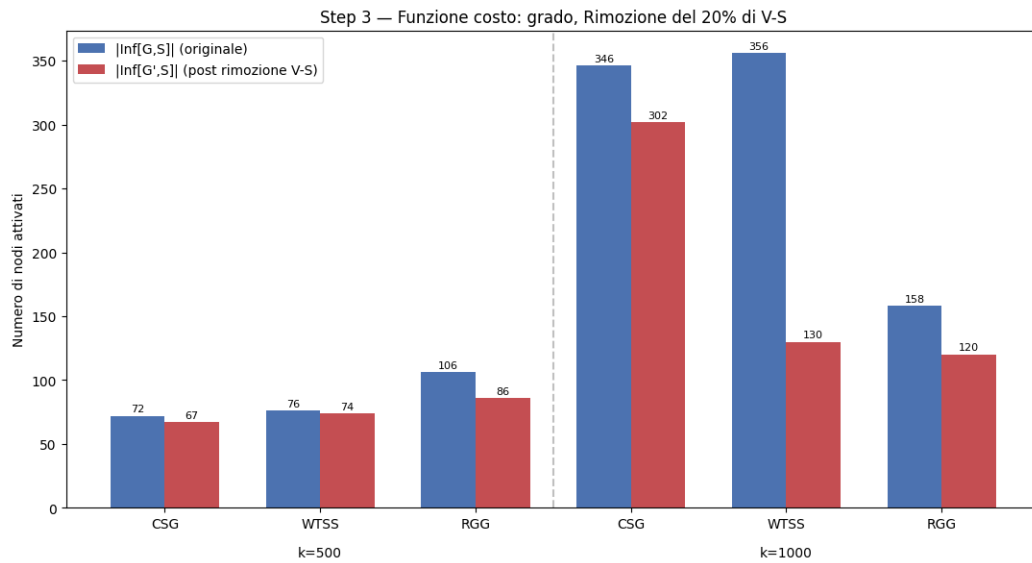


Figure 15: Performance degli algoritmi basata sulla funzione costo grado e sulla rimozione del 20 % di V-S

Anche con la rimozione di vertici il pattern osservato per gli archi si conferma, sebbene con un'intensità complessivamente inferiore ai livelli di perturbazione più bassi. Al 5% la perdita è generalmente contenuta per CSG e RGG (-1.4% e -4.7% a $k = 500$), ma WTSS mostra già un crollo marcato a $k = 1000$ ($356 \rightarrow 188$, -47.2%), a fronte di una perdita quasi nulla a $k = 500$. Al 20% il divario tra algoritmi si amplifica ulteriormente: a $k = 1000$ WTSS perde il 63.5% dell'influenza originale (da 356 a 130 nodi attivati), contro il 12.7% di CSG e il 24.1% di RGG. È interessante notare che, a differenza del caso della rimozione di archi, qui RGG non è sistematicamente il più robusto in termini percentuali a $k = 1000$ (RGG -24.1% contro CSG -12.7%): l'eliminazione diretta dei vertici, anziché delle sole connessioni, colpisce anche i pochi nodi ad alto grado su cui RGG concentra il proprio seed set, riducendo parzialmente il vantaggio strutturale osservato con la perturbazione degli archi. WTSS resta comunque, in ogni configurazione, l'algoritmo più vulnerabile in assoluto.

5.3.2 Funzione costo : ponte

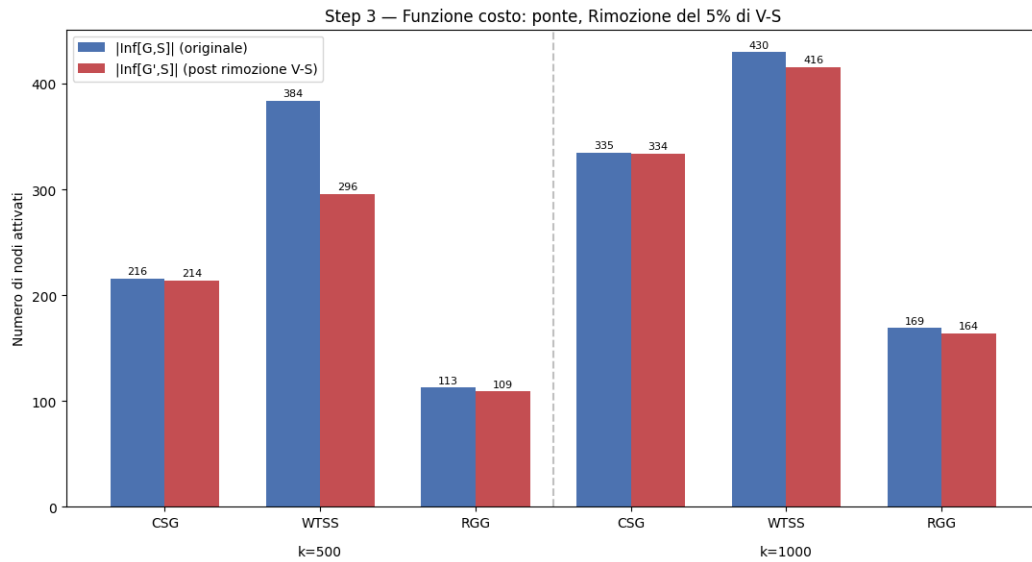


Figure 16: Performance degli algoritmi basata sulla funzione costo ponte e sulla rimozione del 5 % di V-S

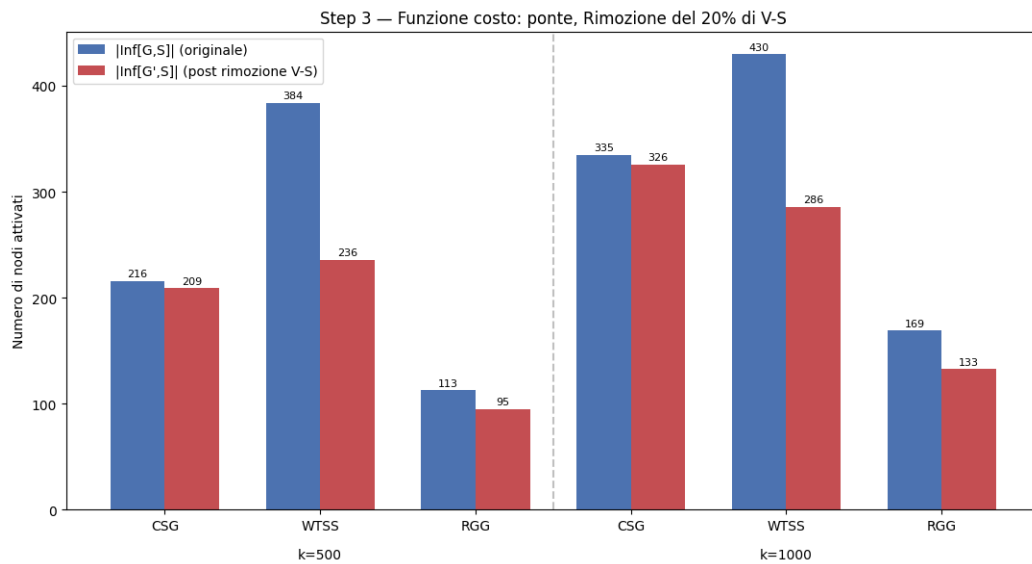


Figure 17: Performance degli algoritmi basata sulla funzione costo ponte e sulla rimozione del 20 % di V-S

Con la funzione costo-ponte l'impatto della rimozione di vertici risulta nel complesso più contenuto rispetto al caso costo-grado, specialmente per CSG e RGG, che perdono al massimo il 3.2% e il 15.9% rispettivamente anche al livello di perturbazione più severo (20%). WTSS si conferma nuovamente l'algoritmo meno stabile, con perdite dal 22.9% (5%, $k = 500$) fino al 38.5% (20%, $k = 500$); a $k = 1000$, tuttavia, la sua fragilità si attenua sensibilmente rispetto al caso costo-grado (−3.3% al 5% e −33.5% al 20%, contro il −47.2% e −63.5% osservati con costo-grado allo stesso budget), suggerendo che, con una distribuzione dei costi che penalizza i nodi-ponte, WTSS tende a selezionare seed set meno concentrati su pochi hub e quindi meno esposti a un singolo evento di rimozione. RGG mantiene, come negli altri scenari, il comportamento più stabile in proporzione al proprio $|Inf[G, S]|$ originale, confermando la maggiore resilienza strutturale del suo criterio di selezione anche di fronte alla perdita diretta di nodi.

6 Conclusioni e Sviluppi Futuri

Il lavoro ha analizzato la selezione di un seed set sotto vincolo di budget in un modello di diffusione a soglia, utilizzando la rete *facebook_combined* di SNAP. Sono stati confrontati tre algoritmi, Cost-Seeds-Greedy, Weighted Target Set Selection e Real-Gain Greedy, combinati con due funzioni di costo, costo-grado e costo-ponte, valutandone sia l'efficacia sia la robustezza a rimozioni di archi e vertici.

I risultati mostrano un compromesso tra efficacia e resilienza. WTSS ottiene generalmente le cascate più ampie sulla rete originale, ma risulta anche il più vulnerabile alle perturbazioni. CSG presenta un comportamento intermedio, mentre Real-Gain Greedy, grazie alla selezione di un pool diversificato di candidati per community, risulta nella maggior parte degli scenari l'algoritmo più robusto in proporzione all'influenza iniziale. Tale vantaggio è principalmente associato al maggiore grado residuo medio dei suoi seed dopo le perturbazioni, sebbene risulti meno evidente nella rimozione diretta dei vertici a budget elevato.

Anche la funzione di costo influenza significativamente il comportamento degli algoritmi. In particolare, il costo-ponte tende a produrre seed set più numerosi per CSG e WTSS e può ridurre, in alcuni scenari, la vulnerabilità di WTSS rispetto al costo-grado.

Sviluppi futuri

I principali possibili sviluppi futuri riguardano:

- **Ripetizione statistica degli esperimenti:** ripetere le perturbazioni con diversi seed casuali e riportare media e deviazione standard delle metriche di robustezza.
- **Estensione dei budget:** analizzare un numero maggiore di valori di k per caratterizzare meglio l'effetto del budget sull'efficacia degli algoritmi.
- **Sensibilità del pool di RGG:** valutare diverse dimensioni e strategie di costruzione del pool per studiare il compromesso tra qualità della soluzione e costo computazionale.
- **Generalizzazione ad altre reti:** replicare l'analisi su reti con caratteristiche topologiche differenti, verificando la generalizzabilità dei risultati, in particolare della robustezza di RGG.